

# **Desarrollo de Software Automatización de Celda de Soldadura**

Zaid Alilou Bouzid  
Francisco Alonso Sastre  
Alejandra Franco Vicente  
Jose Grau Pedrosa  
Roberto Tirado Catalá

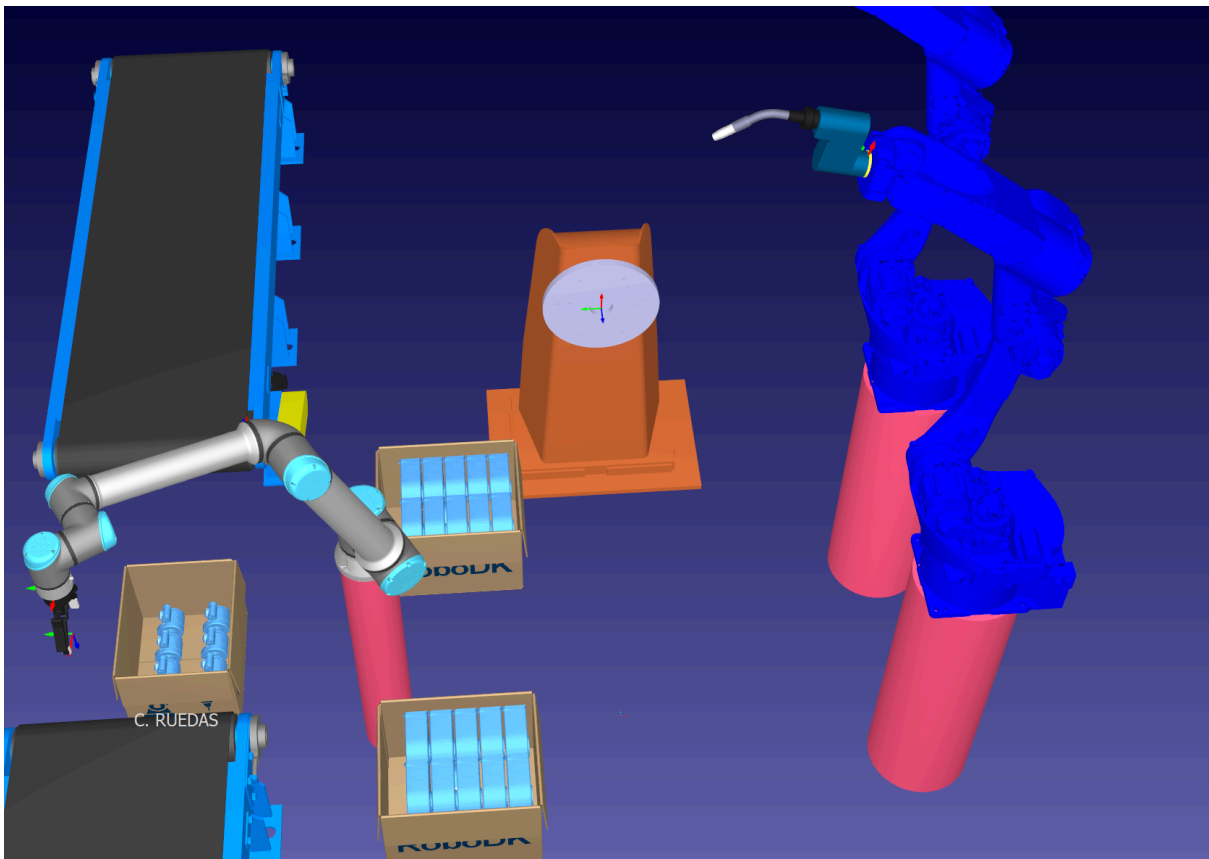
# Índice

|                                                                            |   |
|----------------------------------------------------------------------------|---|
| 1. Introducción y Objetivos.....                                           | 3 |
| 2. Escenarios de Integración.....                                          | 4 |
| Escenario 1: Sistema de Seguridad Perimetral y Parada<br>Sincronizada..... | 4 |
| Escenario 2: Integración MES y Trazabilidad de Inventario....              | 6 |
| 3. Consideraciones de Programación Avanzada (PRA).....                     | 7 |
| 4. Consideraciones de Gestión de Datos Industrial (GDI).....               | 7 |
| 5. Pruebas y Validación.....                                               | 8 |

## 1. Introducción y Objetivos

En este documento definiremos la arquitectura de integración para una celda robótica de soldadura de travesaños. El objetivo principal es pasar de un modelo de automatización aislado a un entorno industrial interconectado mediante eventos (MQTT).

La arquitectura coordina dispositivos perimetrales embebidos (ESP32-S3) con un entorno virtualizado (RoboDK) que gobierna robots de soldadura (Yaskawa AR1440) y manipuladores (UR10e). No solo se controla la producción física, sino que se integra con ámbitos de negocio (Base de Datos).



## 2. Escenarios de Integración

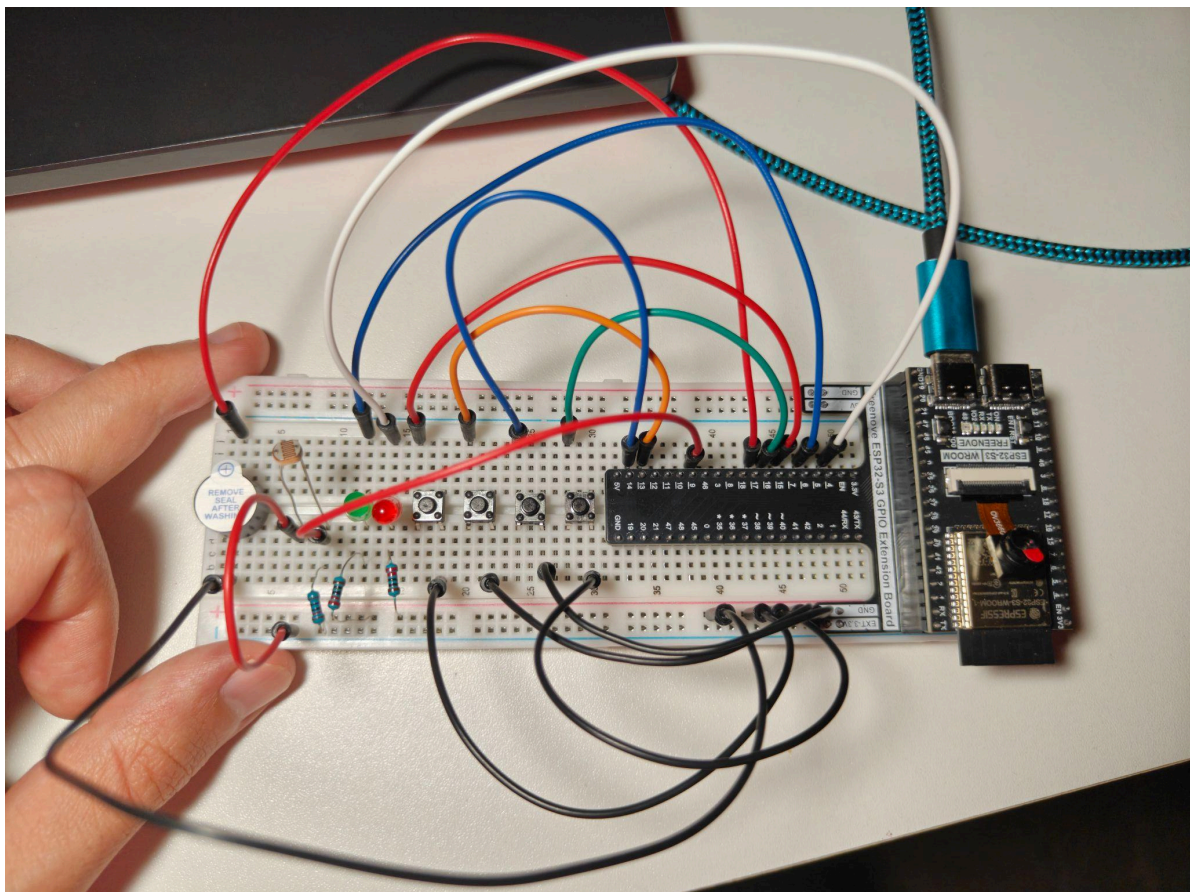
Se han diseñado dos escenarios que comunican la planta física con el entorno de simulación y gestión de recursos.

### *Escenario 1: Sistema de Seguridad Perimetral y Parada Sincronizada*

Este escenario coordina el hardware de seguridad físico con el simulador virtual para gobernar el estado de la celda ante emergencias.

- **Participantes y Roles:**

- ESP32-S3: Dispositivo embebido encargado de la lectura física de sensores (Seta de Emergencia, Barrera LDR) y botoneras. Actúa como controlador de primera línea filtrando ruido y publicando eventos críticos.



- Código de Python RoboDK "Manejador MQTT": Actúa como supervisor lógico. Suscrito al bróker MQTT, procesa los eventos y traslada el estado global a las variables del simulador.

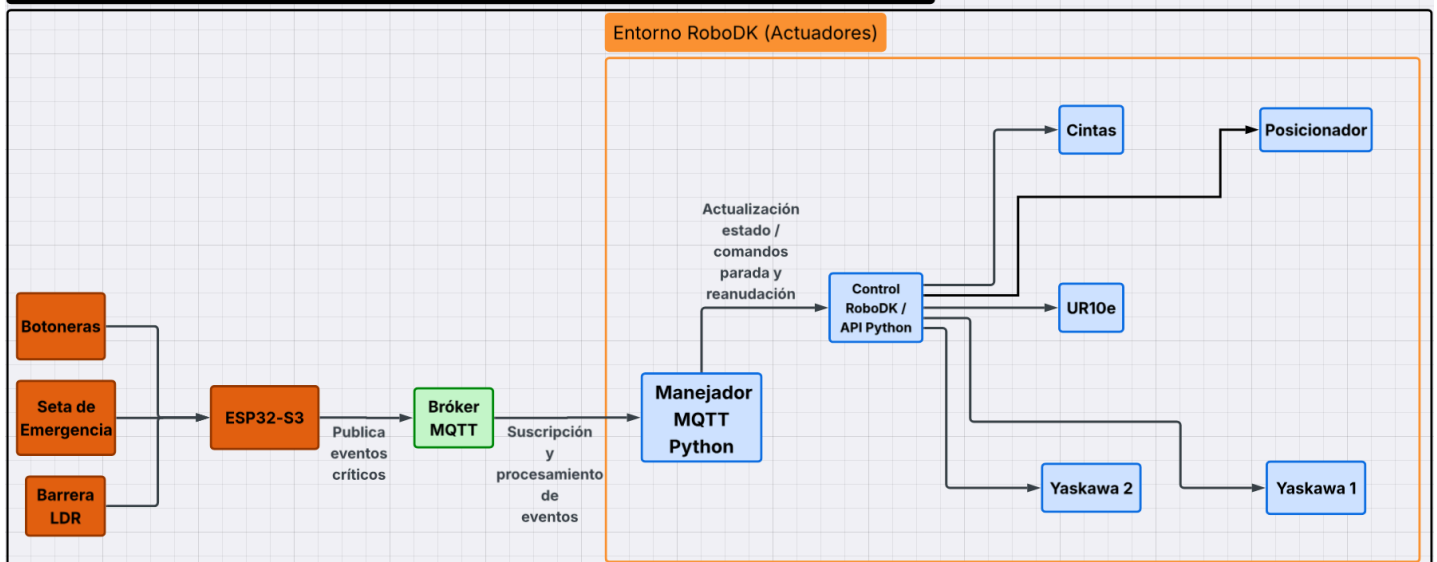
- Entorno RoboDK (Actuadores - UR10e, Yaskawa x2, Cintas): Ejecutan la parada física de los sistemas o su inicio mediante comandos inyectados por la API de Python.

- **Comunicaciones y Mensajería (MQTT):**

- Emisor: ESP32-S3.
- Receptor: Script Python Manejador.
- Topic de comunicación: **celda/seguridad/eventos**
- Formato de Mensaje (JSON):

```
{
    "dispositivo": "barrera_luz",
    "evento": "intrusion",
    "valor": 1250,
    "alarma": true
}
```

#### Arquitectura de flujo horizontal: ESP32-S3, MQTT y RoboDK



## *Escenario 2: Integración MES y Trazabilidad de Inventario.*

Este escenario representa la integración del proceso de automatización con la gestión de recursos de la fábrica. Al finalizar un ciclo de soldadura completo, el sistema lo notifica para descontar los materiales utilizados.

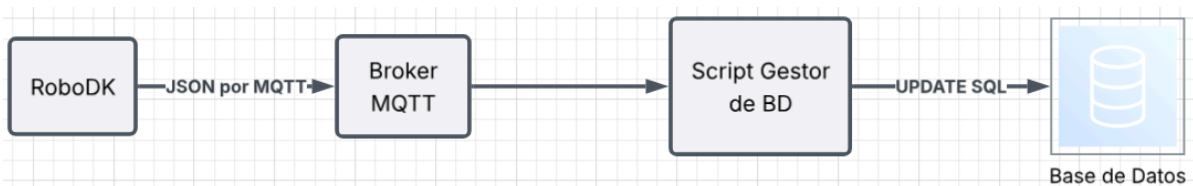
- **Participantes y Roles:**

- Código Python "Manejador MAIN" (Emisor): Vigila la ejecución de los robots en RoboDK. Si el ciclo termina con éxito y el sistema se mantuvo en estado de marcha, publica un evento de validación de pieza.
- Código Python "Gestor de Base de Datos": Aplicación independiente que escucha los eventos de producción, extrae los datos del JSON y ejecuta las sentencias SQL contra el motor de base de datos relacional.
- Base de Datos (GDI): Motor relacional que almacena el stock.

- **Comunicaciones y Mensajería (MQTT a SQL):**

- Emisor: Script Orquestador (RoboDK).
- Receptor: Script Gestor BD.
- Topic de comunicación: **celda/produccion/trazabilidad**
- Formato de Mensaje (JSON):

```
{  
    "evento": "ciclo_completado",  
    "id_pieza": "TRV-001",  
    "cantidad_consumida": 1,  
    "estacion": "celda_yaskawa"  
}
```



Además de mandar mensajes mediante mqtt tiene un almacenamiento local tipo sd, donde se almacenan todos los eventos de una ejecución.

### 3. Consideraciones de Programación Avanzada (PRA)

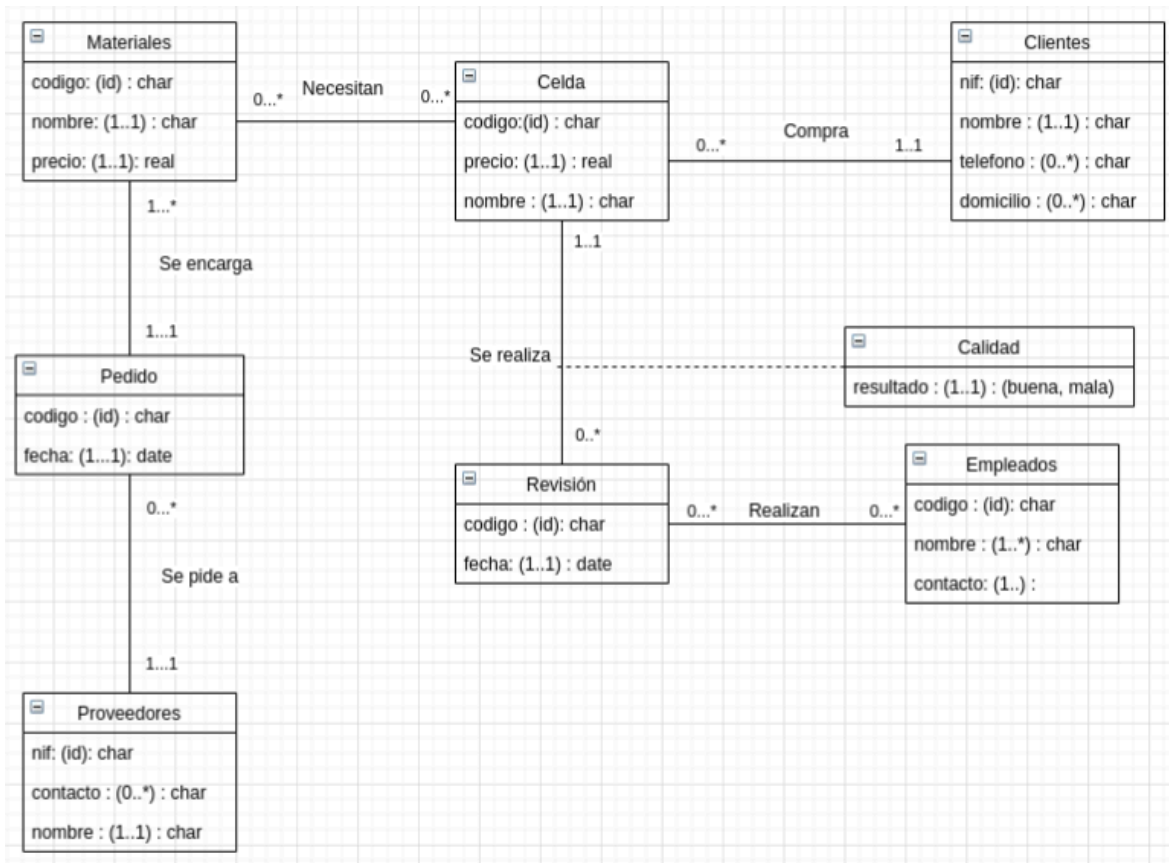
La arquitectura implementada descarta el uso de programación lineal, requiriendo estructuras y algoritmos avanzados para garantizar la seguridad:

- **Estructuras de Datos Complejas:** Se ha implementado el uso de JSON para la serialización y deserialización de datos, permitiendo empaquetar variables heterogéneas en cargas de red ligeras. En RoboDK, se emplean vectores dinámicos (listas) para almacenar los punteros de los objetos actuadores (*activos\_celda* = ["Soldador 1", "UR10e", "Cinta 1"]), permitiendo iterar paradas de emergencia en milisegundos sin replicar código.
- **Algoritmos de Programación Avanzada:** El sistema central está gobernado por una Máquina de Estados Finita integrada en el Manejador (ESTADO\_SISTEMA: ALARMA, ESPERA, MARCHA). Este algoritmo de control concurrente permite evaluar en tiempo real, mediante un bucle infinito (`while prog_main.Busy()`), si el proceso de soldadura es interrumpido por un evento de red asíncrono. Esto evita condiciones de carrera y garantiza que no se lancen ejecuciones simultáneas de colisión.

### 4. Consideraciones de Gestión de Datos Industrial (GDI)

Para simular un entorno real, el proyecto incluye la integración de los datos de fabricación directamente en una base de datos relacional.

- **Esquema de Base de Datos:** Dentro del diagrama Entidad-Relación global del sistema, el proceso de trazabilidad incide directamente sobre la entidad Materiales.



- **Proceso de Integración:** Se ha desarrollado un programa integrador que evita las roturas de stock. Funciona extrayendo los datos recibidos desde el entorno robótico (vía el topic `celda/produccion/trazabilidad`) y ejecutando actualizaciones automáticas. De este modo, la BD refleja el inventario físico real sin intervención humana.

## 5. Pruebas y Validación

Para evaluar la solución implementada, se debe seguir la siguiente secuencia:



1. Arranque del Bróker.
2. Establecer los parámetros en los programas, (tanto en Arduino, en Config.h como en los programas de Python)
3. Entorno Virtual: Abrir la estación de RoboDK con los activos cargados (UR10e, Yaskawa, Cintas) y ejecutar el script **MAIN\_MANEJADOR** para poner el simulador en estado de "ESPERA".
4. Despliegue Físico: Alimentar la ESP32-S3. Comprobar por el Monitor Serie la conexión a la red Wi-Fi y la sincronización NTP.
5. Validación de simulación: Taponar físicamente el sensor LDR. La ESP32 debe lanzar un JSON de intrusión, y el programa de Python detendrá instantáneamente los movimientos de todos los robots virtualizados.
6. Validación de manejo de la BD: Comprobar la base de datos para verificar que el registro de la intrusión y su posterior rearme manual han sido guardados correctamente.